

The Persistent Memory Bundle

By **Thomas Adair**. Marine. DJ/Producer. Systems Architect — shipping across trading systems, music production, agentic tooling, and consumer apps with AI orchestration.

How I keep Claude coherent across model swaps, session death, and long gaps — the substrate everything else in my workflow sits on top of.

A friend got me into day trading in August of 2025. He pitched it the way everyone pitches it — learn this and replace your paycheck — the same rigamarole the course sellers run. I knew that going in. I went anyway, because I'd been circling the idea for a while and this was the first time someone I trusted put a door in front of me. [CONFIRM]

Around the same time I found out people were writing real software with AI. I had never built a program in my life. There was a Pac-Man clone I self-hosted for my kids, and I'm not sure that counts.

Then I learned you could write a strategy in Pine Script, wire it through TradersPost into a prop firm account, and let it trade. That was the lightbulb. Not “I could make money” — I could build something, and it would do the thing while I slept.

What I actually built, for months, was a habit: open a chat, explain everything from the beginning, get some code, close the chat, lose everything, start over tomorrow. None of it was agentic. When the community tore open around OpenClaw I tried it, then MoltBot, then Hermes, which stuck. The tooling kept changing and the loss never did.

I've spent the past year building a live-money trading system and the AI substrate around it. No revenue yet. I'd rather it prove itself before I decide what it becomes.

The one architectural decision I'd defend without hesitating is this: memory does not live inside the model.

Not in the session. Not in the chat window. Not in whichever context length the vendor is advertising this quarter — that number climbs every few months and it has never once been the thing that saved me. Memory lives in files I own, on hardware I control, updated on a discipline, and read by every session that opens.

Three layers do that work. Here's what each one is, and what broke to force it into existence.

Layer one — the local bundle

The first layer is a directory of Markdown files.

```
~/agent/memory/  
├─ MEMORY.md           # the index  
├─ user_writing_voice.md  
├─ feedback_confidence_tagging.md  
├─ feedback_topstep_blownd.md  
├─ project_trident_forge.md  
├─ project_open_orchestra.md  
├─ reference_karpathy_claude_md.md  
├─ notes_agentic_design_patterns_ch11.md  
├─ audit_gap_verification_2026-06-25.md  
└─ ... and a good many more
```

That's it. That's the whole shape. A folder, some text files, and an index at the top that knows where everything is.

I didn't come up with this shape. Andrej Karpathy put out a gist for an LLM wiki — an index file, a pile of small linked articles, wikilinks running between them — and what's on my disk is a direct descendant of it. I read it, recognized it as the answer to a problem I'd been losing to for months, and built my own thing on the bones of his idea. That's a different act than taking someone's work, and I want to be clear which one this is. He gave the shape away for free. I'd rather say his name than quietly absorb it ... so: thank you, Andrej.

What I changed was the typing, and I changed it because my failures were typed.

"The model forgot something" was never one problem. Forgetting who I am, forgetting a correction I'd already given, forgetting where a build stands, forgetting a paper I meant to keep — those are four different failures with four different costs, and if you can't tell them apart you can't fix any of them. So every file carries front-matter with a `type`, and the types *are* the failure classes.

User memories hold who I am and how I want to be spoken to. **Feedback** memories hold corrections I don't want to give twice. **Project** memories hold the state of a build in flight. **Reference** memories hold papers and repos with my note about *why* each one matters — the why is the part that makes it a tool instead of a bookmark. **Notes and audits** hold the thinking work.

Every file has one job. Reads like a note from me to a future me. Small enough to open inside a single request.

The index at the top is the concierge. One line per entry — `[Title](file.md) - one-sentence hook` — and every session opens it first. If nothing in there is relevant, the session goes on about its business. If something is, it opens that one file and reads only what matters. The index doesn't hold the memories. It holds the map.

The whole layer is deliberately old-fashioned. Filesystem, plain text, greppable. It isn't clever and it doesn't need to be. It needs to survive the death of the session that wrote it and be readable by the next one ... and it does both, which is more than I can say for anything else I tried.

Layer two — the brain

The second layer exists because the first one is only as portable as the machine it's sitting on.

The bundle solved session death. It did not solve me. I work across a desktop and a laptop, and the sessions that mattered were never reliably on the machine holding the files. I'd walk into the other room, open a session, and be right back where I started — explaining myself to a model that had every reason to know better.

I already had a VPS running the trading system around the clock. The durable box was sitting there either way. Putting shared memory on it was less a design decision than noticing the obvious.

So on that VPS I run a small service — a durable, queryable record of decisions, lessons, and state across every project I touch, not just the one on the laptop in front of me — exposed over MCP. Any session, anywhere, can call `brain_digest()` for what's happened lately across the whole ecosystem, or `brain_context("trident-forge")` for what's current on one system.

That's the thing that turns a local folder into an actual substrate. On the laptop the local bundle is thin. The brain is the same brain. The session reconstructs its bearings from the shared source and gets on with it. When a session ends and the transcript evaporates, what it *decided* outlives it.

The brain has its own failure modes — there's a token-drift edge I run a reconciliation script against — but it's built fail-soft on purpose. If the brain is unreachable, sessions fall back to the local bundle and log a warning. Nothing blocks. A memory system that can halt the work is a memory system I'd stop using by Thursday.

Layer three — the discipline

The third layer is the one people underestimate, and it's the one that actually decides whether the other two are worth anything.

Files and a server do not create memory. What creates memory is writing things down at the moment they happen, in the shape that will be readable later, at a cadence that keeps up with the work. I learned that the way you learn most things — by having files that were technically present and quietly wrong, and by trusting them.

Four rules govern the cadence. Every one of them is scar tissue.

One. The moment a session corrects a class of behavior, a feedback memory gets written. Not later. If I have to correct the same thing twice, the second correction is a bug in my bundle, not a bug in the model. That reframe is the single most useful thing in this essay — it turned correcting the AI from a chore I resented into maintenance on a system I own.

Two. The moment a project decision locks, the project file gets updated. Decision, date, current-truth prose. Skip it and the next session picks up a plan I already abandoned and executes it faithfully, at speed, with total confidence. There is nothing quite like watching a machine do excellent work on last week's plan.

Three. The moment a paper, a repo, or a finished piece of work earns a second look, it gets saved in that same session. I finished a substantial essay once and wrote no memory for it. Days of sessions had no idea it existed. I had to re-explain a *finished thing* to my own system, which is a particular kind of humbling. A reference I mean to save "next time" is a reference I have already lost.

Four. Every few substantive exchanges, the session checkpoints anything worth surviving. Added the day I lost an afternoon to a session that dropped context without warning and kept talking like nothing had happened.

Without the discipline, the files exist and they lie. With it, the files are current-truth, and the whole substrate carries weight.

What this bought me

Once memory lives outside any individual session, everything downstream gets cheaper.

Model routing gets cheaper. A lighter, faster model can take a targeted job without re-learning who I am — because who I am is a file it can open. When live-money code needs the

strongest model I have, I route deliberately. That routing rule lives in the bundle too, which means I don't have to remember to remember it.

Long projects get cheaper. A week away no longer costs a re-briefing. The audio-plugin work, the ebook reader, the visual-effects rig I use for DJing, the trading engine — all of them sit in the substrate at once, each one callable without dragging the others into context.

Long conversations get cheaper. When a session nears its ceiling, the handoff rule fires, it writes a handoff file, and the next session picks the thread up mid-sentence.

The writing you're reading got cheaper. I didn't have to remember how I wanted this to sound. A `user_writing_voice.md` file handed the drafting session the register to reach for — written once, from a calibration I only had to do once.

And it compounds. Every deep read lands as a reference or a note before the session closes. Every rule I lock lands as feedback. What I've learned this past year isn't scattered across dead transcripts. It's consolidated, cross-linked, and queryable — which is a sentence I could not have written a year ago about anything I owned.

What it costs

Nothing this useful is free. Three bills come due.

Pruning against rot. A rule that was right in spring can be wrong by summer. If old memories don't get corrected or retired, future sessions follow them dutifully and produce work that is confidently obsolete — the worst failure mode in the whole system, because it *looks* like competence. A weekly pass is how I fight it.

Compacting the index. The whole thing rests on the index being small enough to read in one shot. Past a certain size the tail entries slide out of the read window, and nothing errors, and nothing warns you. The session simply stops seeing your newest memories and carries on. I hit that ceiling again recently. It is a silent failure and it will be a silent failure the next time too.

Reconciling contradictions. A feedback rule and a project decision can quietly disagree with each other. The verify-gate catches some before they reach code. The weekly pass catches more. Neither catches everything, and I know it. A graph-resolver would be the real answer ... someday it'll exist.

The substrate doesn't stop working when these accrue. It works *less well* until they're paid. Gardening, not firefighting.

Why the architecture holds

The best proof I have is completely unremarkable, which is exactly the point.

I ran a machine-health diagnostic one week, wrote down what I found, and then did nothing about it. Life happened. A week later a different session, on the far side of that gap, opened the file, knew precisely what state the machine had been left in, and picked the work up mid-thought. Nothing heroic occurred. Something simply did not have to be re-explained.

That's the whole return. Not a demo, not a benchmark — an absence. The re-briefing that didn't happen.

The three layers are what make it unremarkable. The local bundle keeps per-machine detail close and greppable. The brain keeps cross-machine truth durable and portable. The discipline keeps both of them honest. Take any one away and the other two degrade into a folder of plausible-looking lies.

If you're building alone with AI across anything longer than a single sitting, the answer to "*how do I keep this coherent*" is not a bigger model. It's not a longer context window and it's not a smarter chat UI. It's a memory system that lives outside any individual session, that you own end to end, and that you write to on a discipline you actually keep.

It doesn't have to be Markdown. It doesn't have to be a VPS. It does have to survive session death, and it does have to be *read* by the next session — not merely written for you.

The rest of what I build sits on top of this.

More field notes to follow.

— *Thomas Adair. Marine. DJ/Producer. Systems Architect — shipping across trading systems, music production, agentic tooling, and consumer apps with AI orchestration.*